# CSE 4303
# Data Structures

Topics: Time Complexity, Algorithm Analysis, Asymptotic Notations

Sabbir Ahmed
Assistant Prof | CSE | IUT
sabbirahmed@iut-dhaka.edu

# Data Structure

- The **Logical or mathematical model** of a particular organization of data.
- Data structure is a **way of organizing data** in a computer so that it can be used effectively to solve specific problems.
  - For example, we can store a list of items having the same data-type using the '*array data structure*'.

Logical / Mathematical Model particular org.

- Which one is the best data structure??
- Data structure operations:
  - Traversing
  - Searching
  - Inserting
  - Deleting
  - Sorting
  - Merging

  And so on…

  Big questions:
  - How do we determine the **efficiency** of an algorithm?
  - How to choose between multiple alternatives?

# Complexity, Time-space tradeoff

- Function to estimate running **time/space** with respect to the **input size**.
- Less time and space requirement is a blessing!
- Deals with large input size.
- Tradeoff: Increased amount of space to store data can sometimes reduce time requirement (or vice-versa).

# Time complexity – why do we care?

Let's check if an integer is prime or not!

**solution#1**

```
for i=2 to n-1
   if i divides n
        n is not a prime
```

$(n-2)$ divisions in worst case

**solution#2**

```
for i=2 to √n
   if i divides n
        n is not a prime
```

$(\sqrt{n}-1)$ divisions in worst case

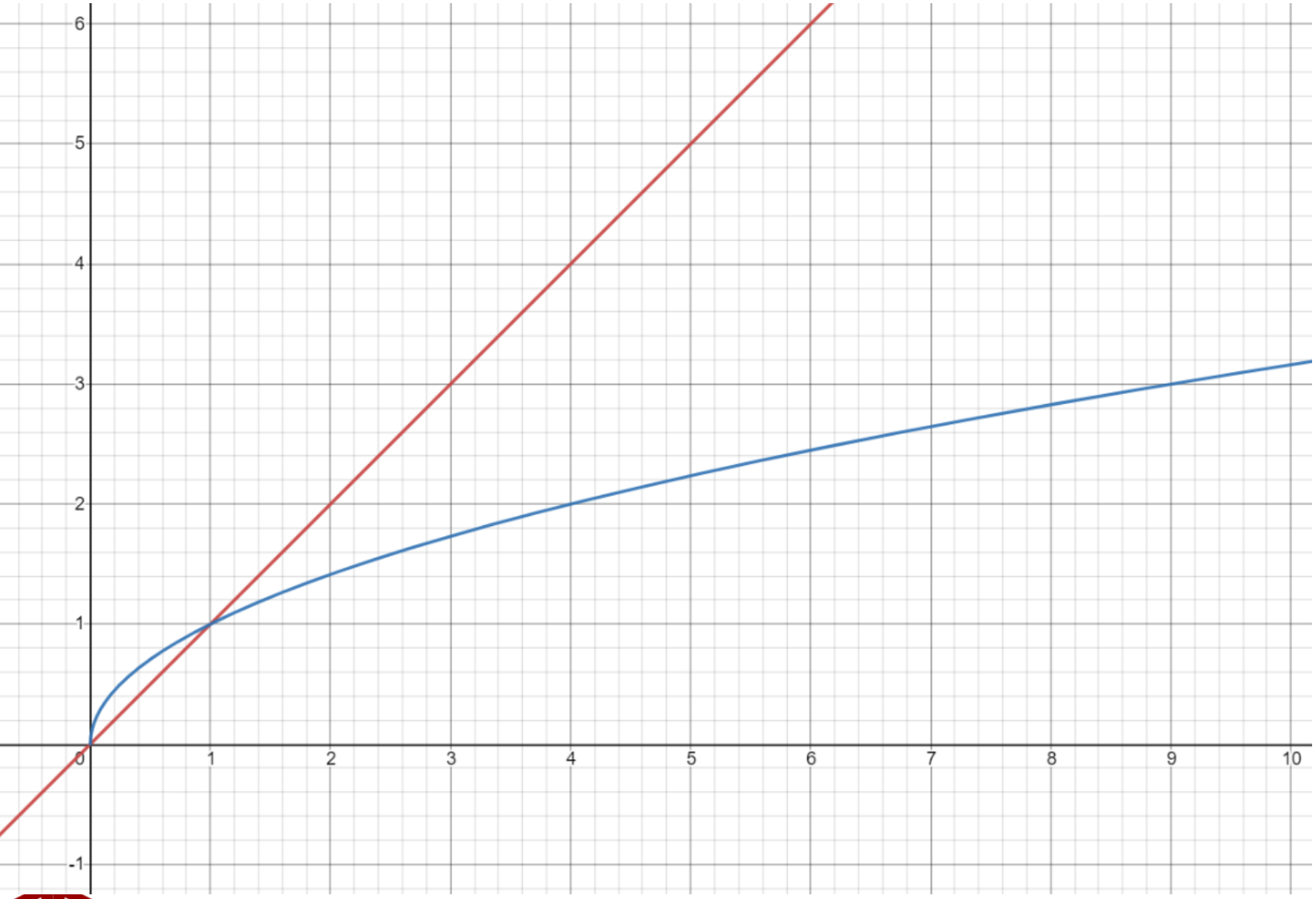
| Assuming 1 ms to perform a division | | |
|---|---|---|
| | Solution #1 | Solution#2 |
| n=11 | 9 ms | ~2 ms |

| Assuming 1 ms to perform a division | | |
|---|---|---|
| | Solution #1 | Solution#2 |
| n=11 | 9 ms | ~2 ms |
| n=101 | 99 ms | ~9 ms |

| Assuming 1 ms to perform a division | | |
|---|---|---|
| | Solution #1 | Solution#2 |
| n=11 | 9 ms | ~2 ms |
| n=101 | 99 ms | ~9 ms |
| n=1000003<br>=10^6+3 | ~10^6 ms =1000sec<br>=16.66min | ~10^3 ms<br>= 1sec |

| Assuming 1 ms to perform a division | | |
|---|---|---|
| | Solution #1 | Solution#2 |
| n=11 | 9 ms | ~2 ms |
| n=101 | 99 ms | ~9 ms |
| n=1000003<br>=10^6+3 | ~10^6 ms =1000sec<br>=16.66min | ~10^3 ms<br>= 1sec |
| n=10^10 | 10^10 ms =10^7sec<br>=115 days | ~10^5 ms = 100sec<br>= 1.66 mins |

| Assuming 1 ms to perform a division | | |
|---|---|---|
| | Solution #1 | Solution#2 |
| n=11 | 9 ms | ~2 ms |
| n=101 | 99 ms | ~9 ms |
| n=1000003<br>=10^6+3 | ~10^6 ms =1000sec<br>=16.66min | ~10^3 ms<br>= 1sec |
| n=10^10 | 10^10 ms =10^7sec<br>=115 days | ~10^5 ms = 100sec<br>= 1.66 mins |
| | T ∝ n<br>AKA. $O(n)$<br>AKA. 'Big-oh of n' | T ∝ $\sqrt{n}$<br>AKA. $O(\sqrt{n})$<br>AKA. 'Big-oh of '$\sqrt{n}$' |




Two functions plotted in this graph:

$f(x) = x$ (red)

$f(x) = \sqrt{x}$ (blue)

Blue function is a bit costly in the beginning, but cheaper as $x$ increases.

# Time complexity analysis

- Measures how fast the time requirement of a program grows *when the input size increases*.

- From the previous slide, note that:
  - Just solving the problem isn't enough!
  - Program must behave well for large input sizes!

- Running time of program may depend on:
  - Single vs multi processor
  - Read/write speed of memory
  - 32-bit or 64-bit
  - Size of input

*For time complexity analysis, we are only interested in #04 (size of input)*

```
Sum(a,b){
   return a+b
}
```

Time requirement: ~ 2 time-units
(1 unit for addition, 1 unit for return statement)

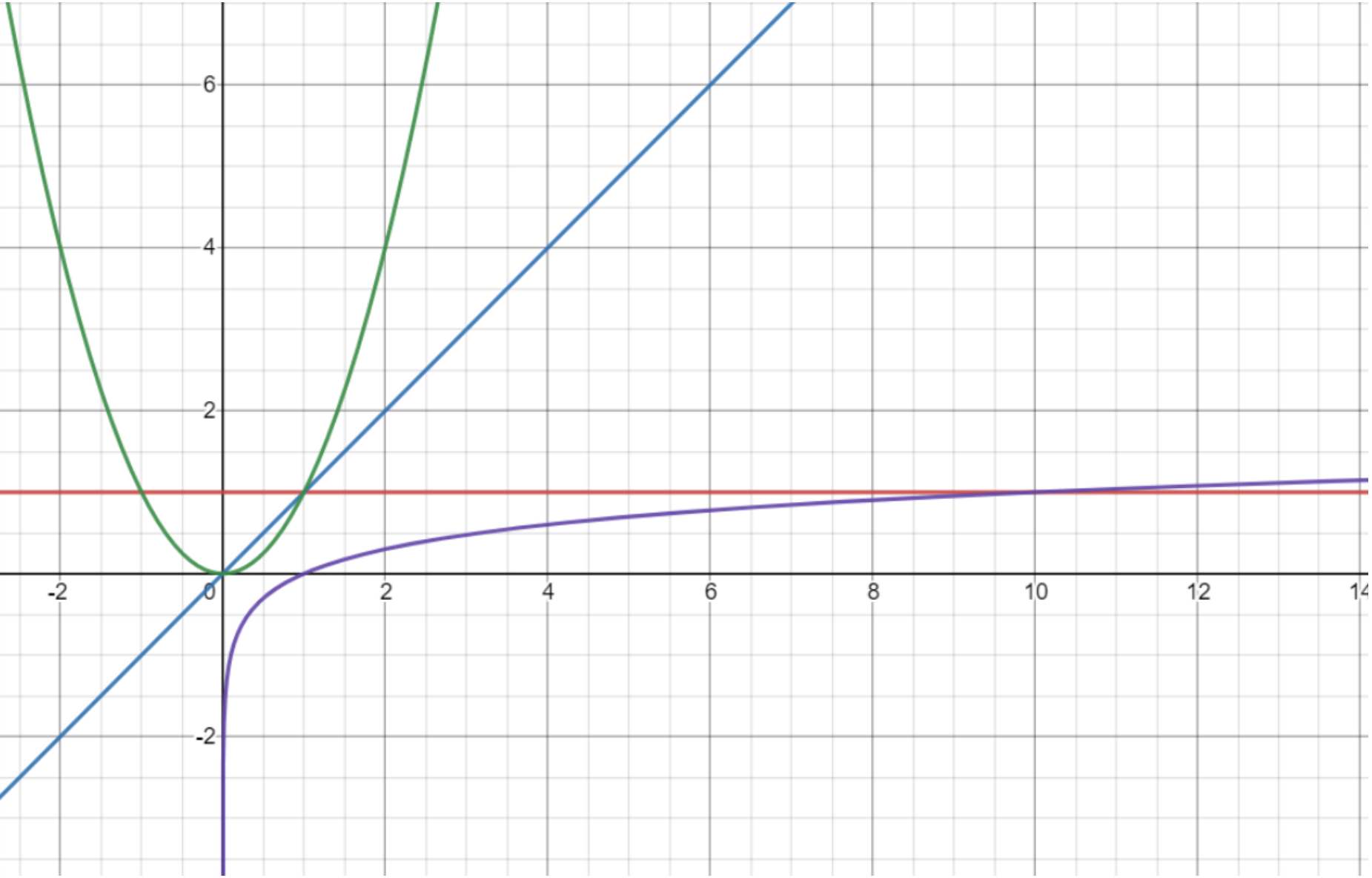
- Takes same amount of time regardless the size of input.
- '*Constant time algorithms*'
- Time complexity: $\boldsymbol{O(1)}$

| | # times | Cost |
|---|---|---|
| 1.sumofList (A,n){<br>2. total=0<br>3. for i=0 to n-1<br>4. total = total + A[i]<br>5. return total<br>6. } | <br>1<br>n+1<br>n<br>1 | <br>1 (c1)<br>2 (c2)<br>2 (c3)<br>1 (c4) |

| Comments |
|---|
| In line 3:<br>- Executes n+1 times. One extra checking for breaking condition.<br>- c2: 1 unit for increment, 1 unit for assignment.<br>In line 4:<br>- 1 unit for addition, 1 for assignment. |

- $T(n) = 1 + 2(n+1) + 2n + 1 = 4n + 4$
- In other words, $\mathrm{T(n)} = c_1 + c_2(n+1) + c_3 n + c_4 = \boldsymbol{cn} + \boldsymbol{c'}$
  - here $(c = c_2 + c_3\ ,\&\ c' = c_1 + c_3 + c_4)$
- Don't care much about value of $c\ or\ c'$, focus on the rate of growth.
- Here the growth is linear. Termed as ***O(n)***, AKA 'Big-oh of n' AKA 'Order of n' .

| Goal | Time taken | Notation | Growth with input |
|---|---|---|---|
| Adding two numbers | $T(n) = constant$ | $O(1)$ | No growth |
| Adding all numbers in a list | $T(n) = cn + c'$ | $O(n)$ | Linear |
| Adding all numbers in a 2D-matrix | $T(n) = an^2 + bn + c$ | $O(n^2)$ | Quadratic |




$f(x) = 1$ (red),

$f(x) = x$ (blue),

$f(x) = x^2$ (green),

$f(x) = logx$ (purple)

Check the growth of function as values in x axis grows!

We analyze time complexity for:

- Very large input-size
  - Compare two functions $T(n) = n^3 + 3n^2 + 4n + 2$ & $T(n) = n^3$
  - Might look different for smaller values of '$n$', but behave similarly for larger $n$.

- Worst case scenario
  - E.g. while searching a number out of '$n$' elements, the number might be found within the first few places, but not all the time!

# Algorithm Analysis

- Goal of algorithm analysis is to *take a block of code* and determine the *asymptotic run time* or *asymptotic memory requirements* based on various parameters.

- The *order of growth of running time* of an algorithm gives:
  - A simple characterization of the algorithm's efficiency
  - Allows to compare the relative performance of alternative algorithms.

- Given an array(input) of size $n$:
  - 'Selection sort' requires $\theta(n^2)$ time
  - 'Merge sort', 'Quick sort' and 'Heap sort' all require $\theta(nlogn)$ time

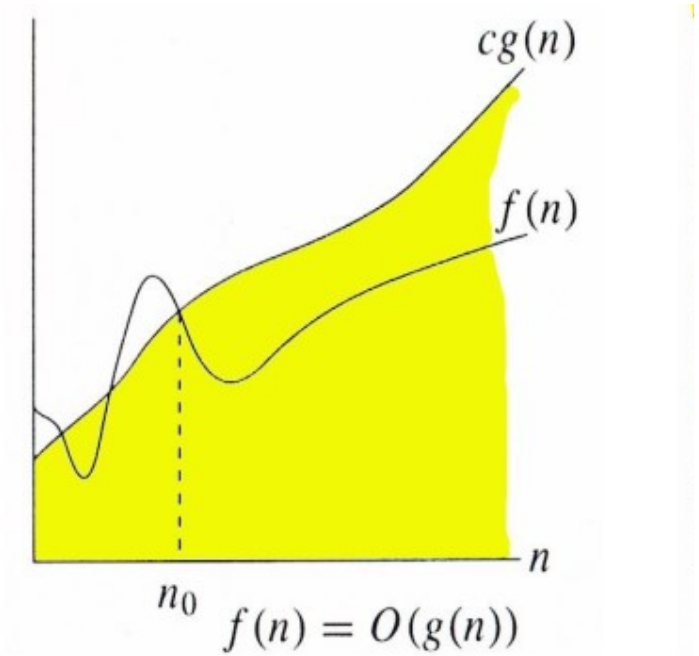
- However:
  - Merge sort requires $\theta(n)$ additional memory
  - Quick sort requires $\theta(logn)$ additional memory
  - Heap sort requires $\theta(1)$ memory

# Asymptotic Analysis

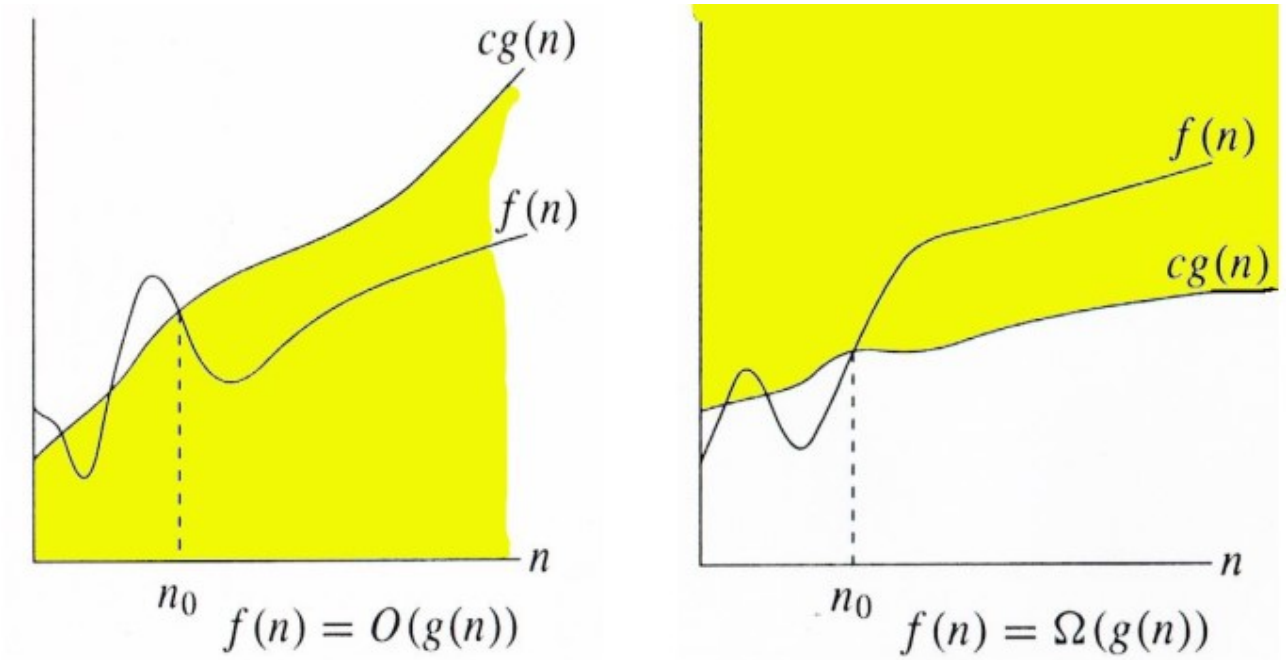
- *Asymptotic Analysis* is the big idea that helps to analyze algorithms.
  - In Asymptotic Analysis, we evaluate the performance of an algorithm in *terms of input size* (*we don't measure the actual running time*).
  - Define *mathematical bound* of how the time (or space) taken by an algorithm increases with the input size.

- An algorithm that is *asymptotically more efficient* will be the best choice for all but very small inputs.

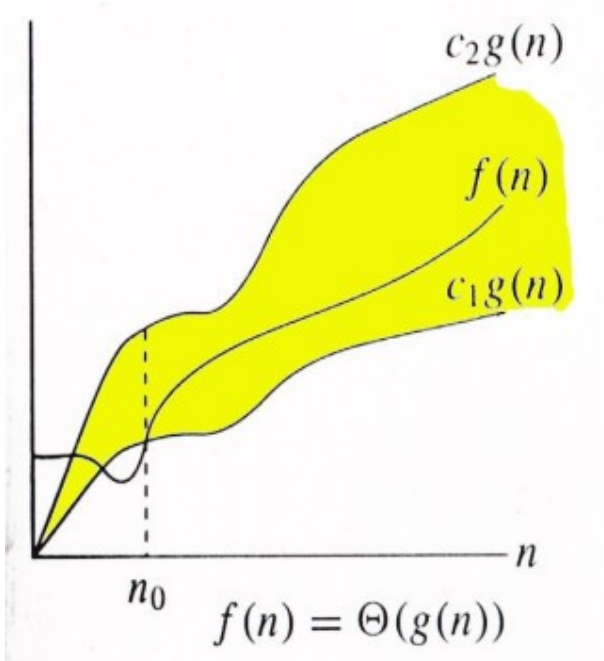
[Generally, the term '**asymptotic**' means *approaching but never connecting* with a line or curve.]

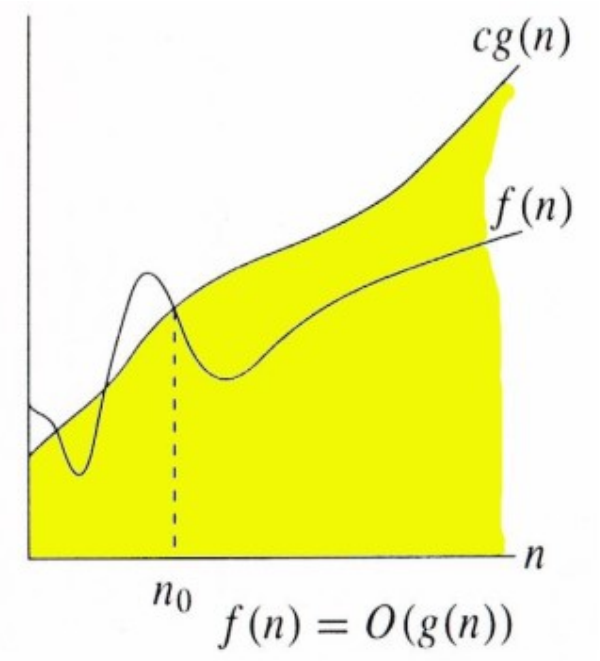


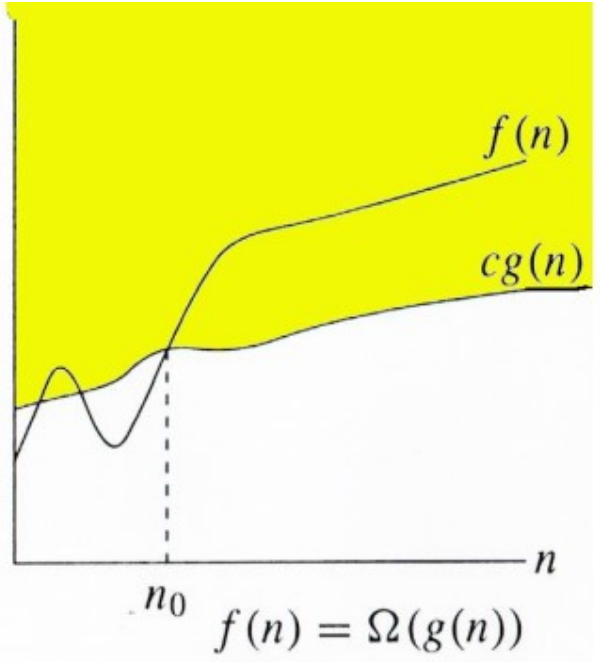

(Relations between $\theta, O, \Omega$)




(Relations between $\theta, O, \Omega$)



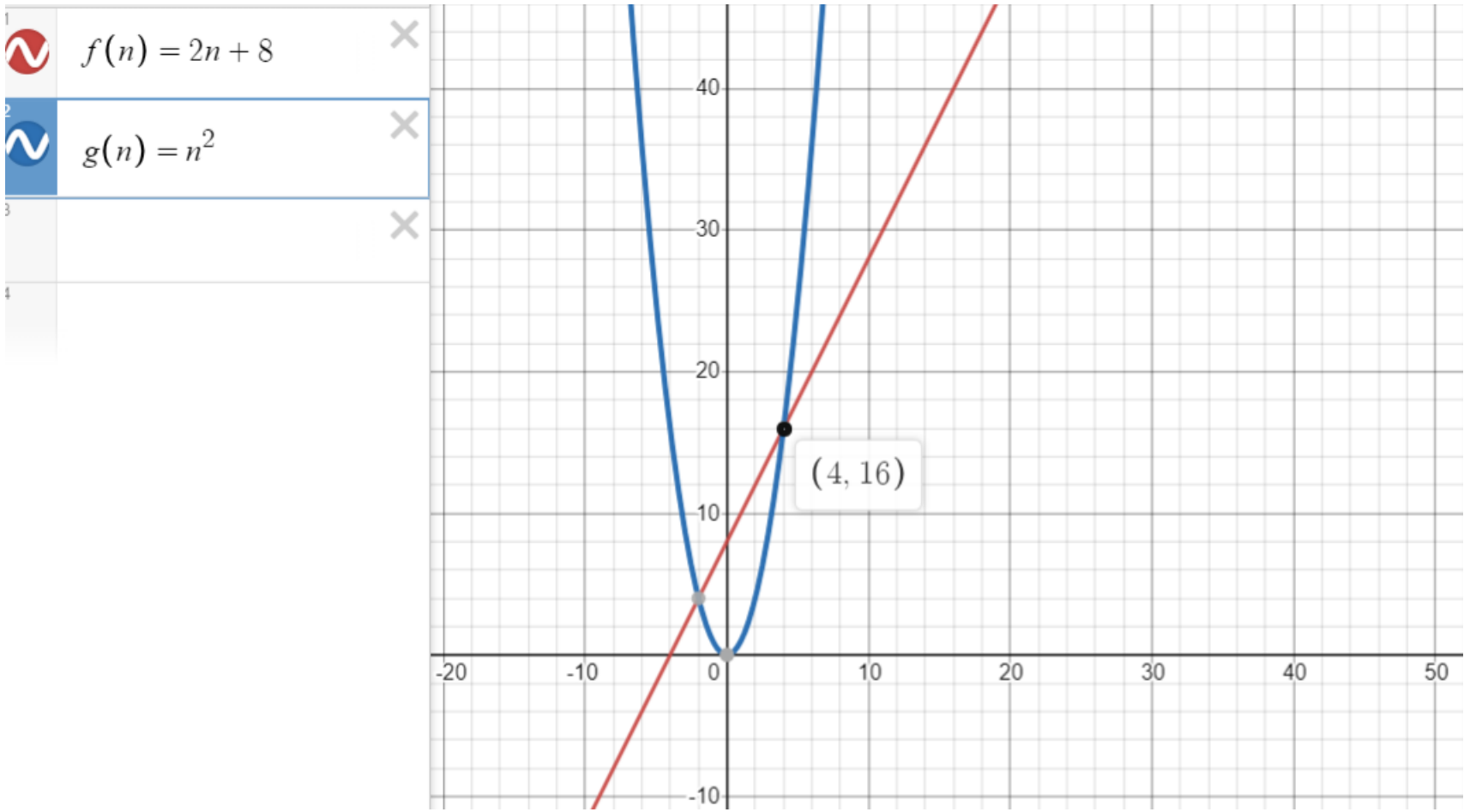




(Relations between $\theta, O, \Omega$)

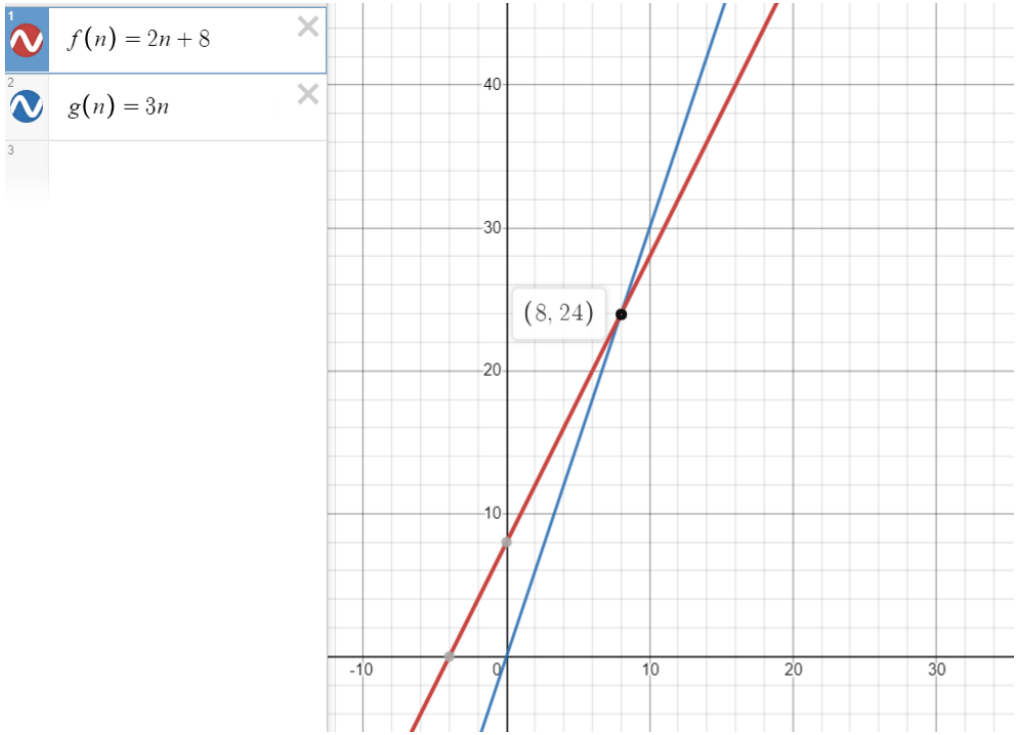
- The $'O'$ Notation
  - A function $f(n) = O\big(g(n)\big)$ if there exists $n_0$ $and$ $c$ such that $f(n) < cg(n)$
  - Whenever, $n > n_0$
    - $O$ (pronounced big-oh) is the formal method of expressing *upper bound* of an algorithm's running time.
    - Measures the *longest amount of time it could possibly take*.
    - $g(n)$ is an *asymptotic upper bound* for $f(n)$.

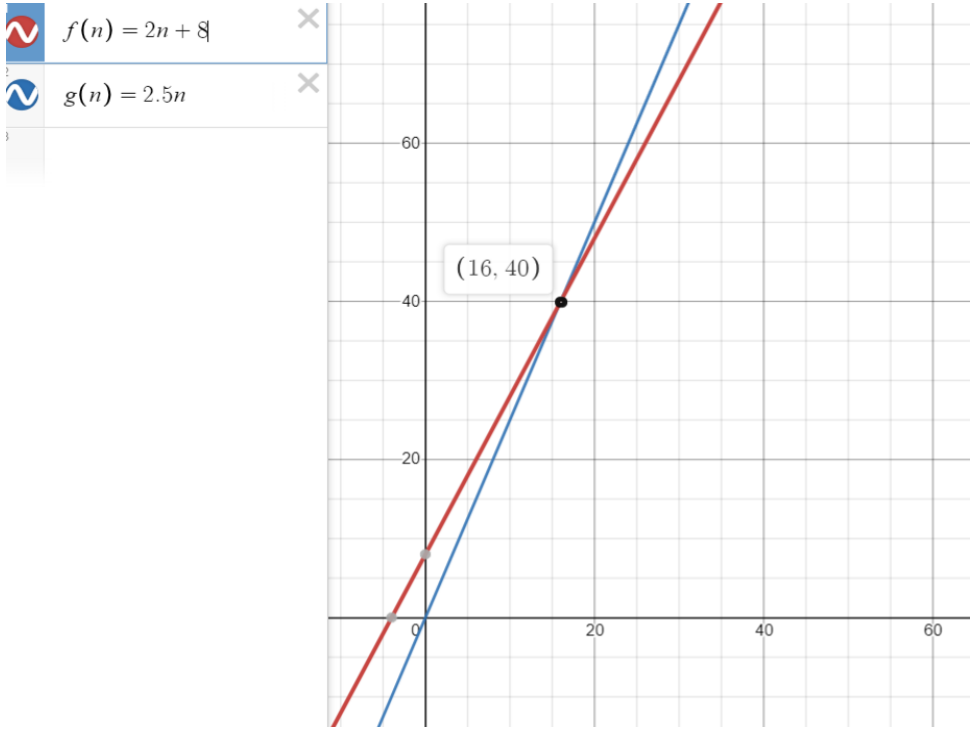
- Example of $'O'$ notation:
  - Suppose, $f(n) = 2n + 8$ and $g(n) = n^2$
  - Can we find a constant $n_0$, so that $2n + 8 \leq n^2$?
  - $n_0 = 4$ works here!
  - For any number $n$ greater than $4$, this will still work. Since we are trying to generalize this for large values of $n$
    - $f(n)$ is bounded by g$(n)$ and will always be less. (here $c = 1$ is good enough.)
    - Conclusion, $f(n) = O\big(g(n)\big),\ for\ all\ n > 4$
    - Thus here, $f(n) = O(n^2)$



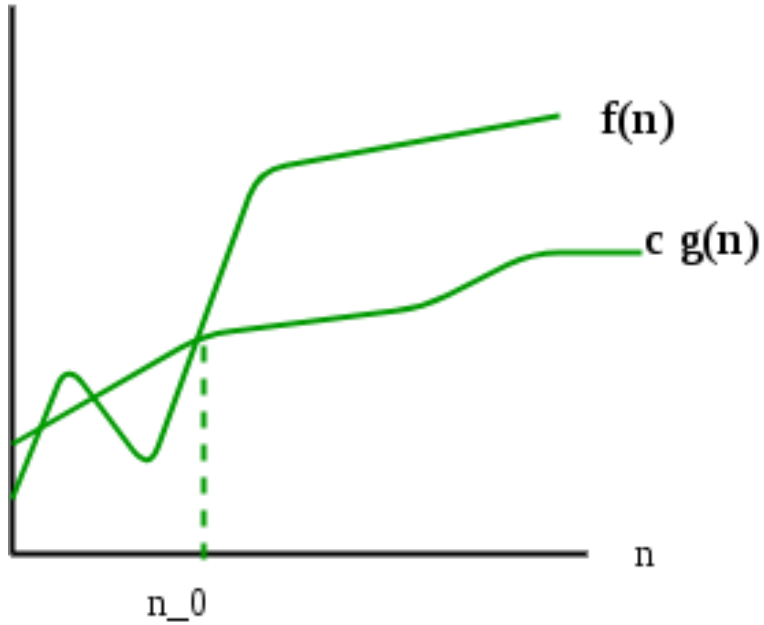

$f(n) = 2n + 8$
$g(n) = n^2$
(4, 16)
40
30
20
10
-10
-20
-10
0
10
20
30
40
50

- Can we bound $f(n) = 2n + 8$ using $g(n) = n$ ? (meaning, can $f(n) = O(n)$ be true? )
  - Yes! Pick the value of $'c'$ carefully!
  - $if\ c = 3,\ f(n) = O(n)\ for\ all\ n \geq 8$
  - We can also define, $if\ c = 2.5,\ f(n) = O(n)\ for\ all\ n \geq 16$



- Big-Omega Notation:
  - A function $f(n) = \Omega(g(n))$ if there exists $n_0 \; and \; c$ such that $f(n) > cg(n)$
  - Whenever $n > n_0$:
    - Almost same definition as Big-Omega, except that $'f(n) > cg(n)'$
    - This makes $g(n)$ a *lower bound* function, instead of a upper bound function.
    - $g(n)$ is an *asymptotic lower bound* for $f(n)$
    - Describes the *best that can happen* for a given data size.

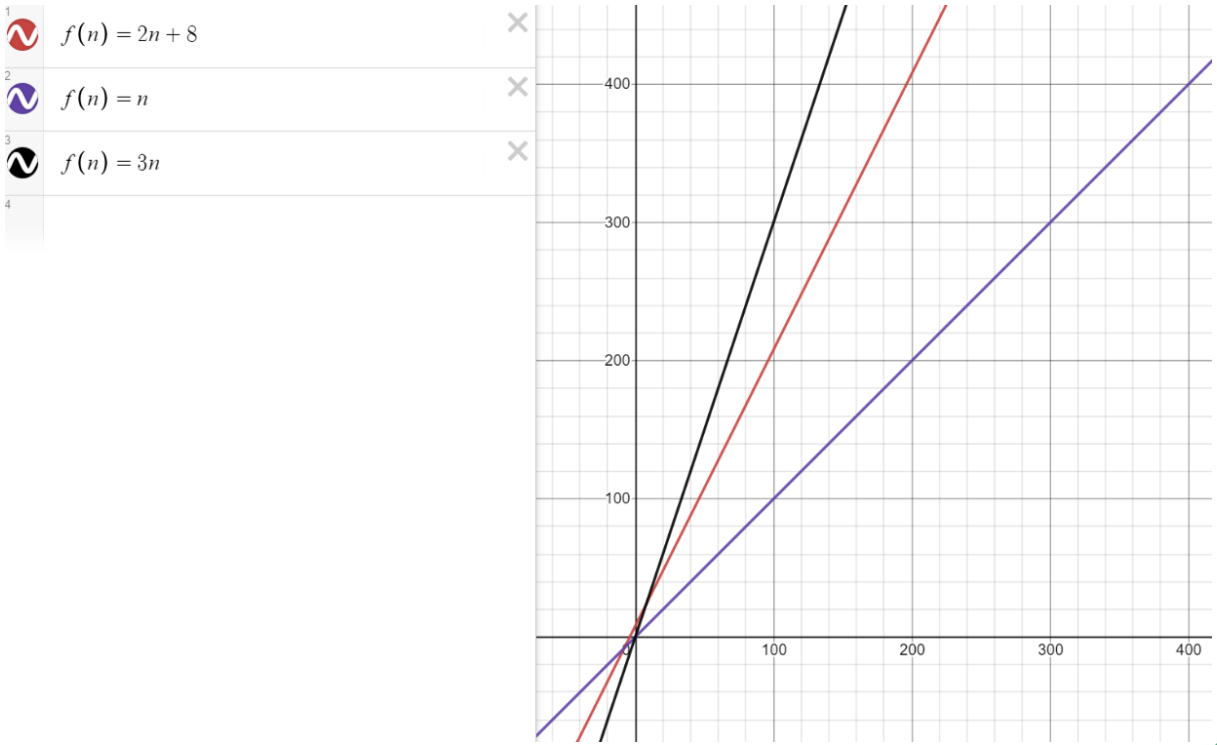


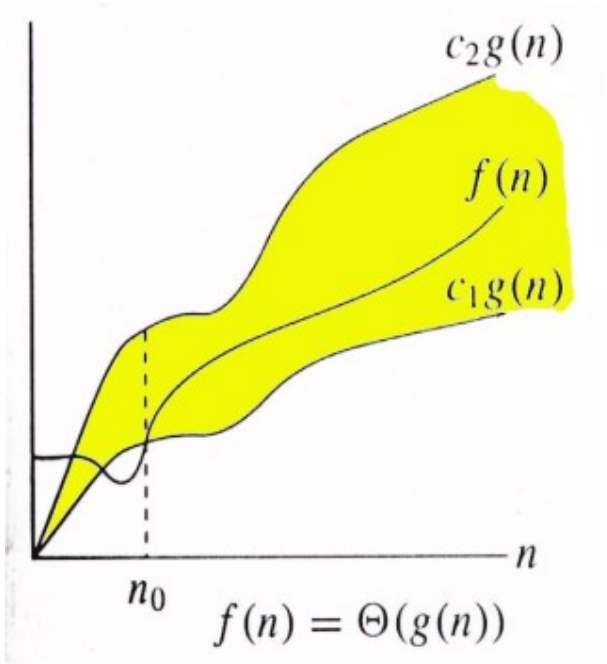

f(n) = Omega(g(n))

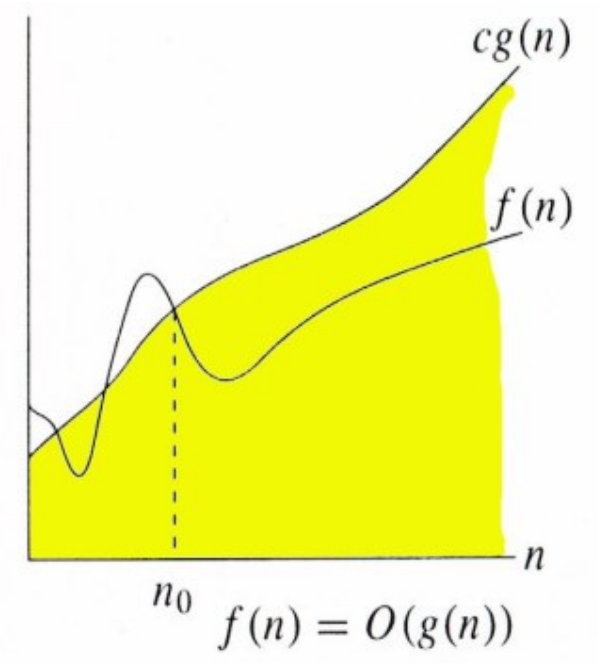
- Big-Theta Notation:
  - A function $f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$
  - $f(n)$ is *bounded both from the top and bottom* ***by the same function*** $\boldsymbol{g(n)}$.
  - Thus, $g(n)$ is an *asymptotic tight bound* for $f(n)$
  - Tight bounds are obtained from asymptotic upper and lower bounds.

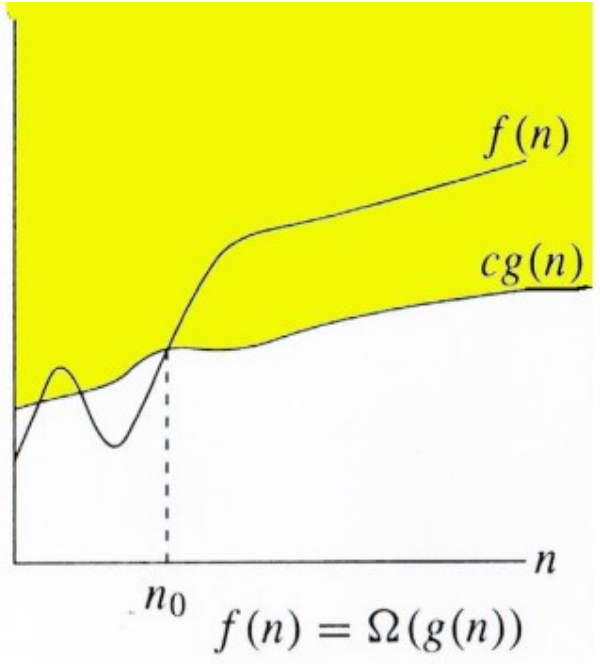
  - $3n + 3$ is:
    - $O(n)$ (let's say for $c = 4$)
    - $\Omega(n)$ (let's say for $c = 1$)
    - So it can be written as $\Theta(n)$

  - $3n + 3$ is
    - $O(n^2)$ $(for\ all\ n \geq 4)$
    - ~~$\Omega(n^2)$~~ (only true for $n = 1,2,3$)
    - So it **can not** be written as ~~$\Theta(n^2)$~~





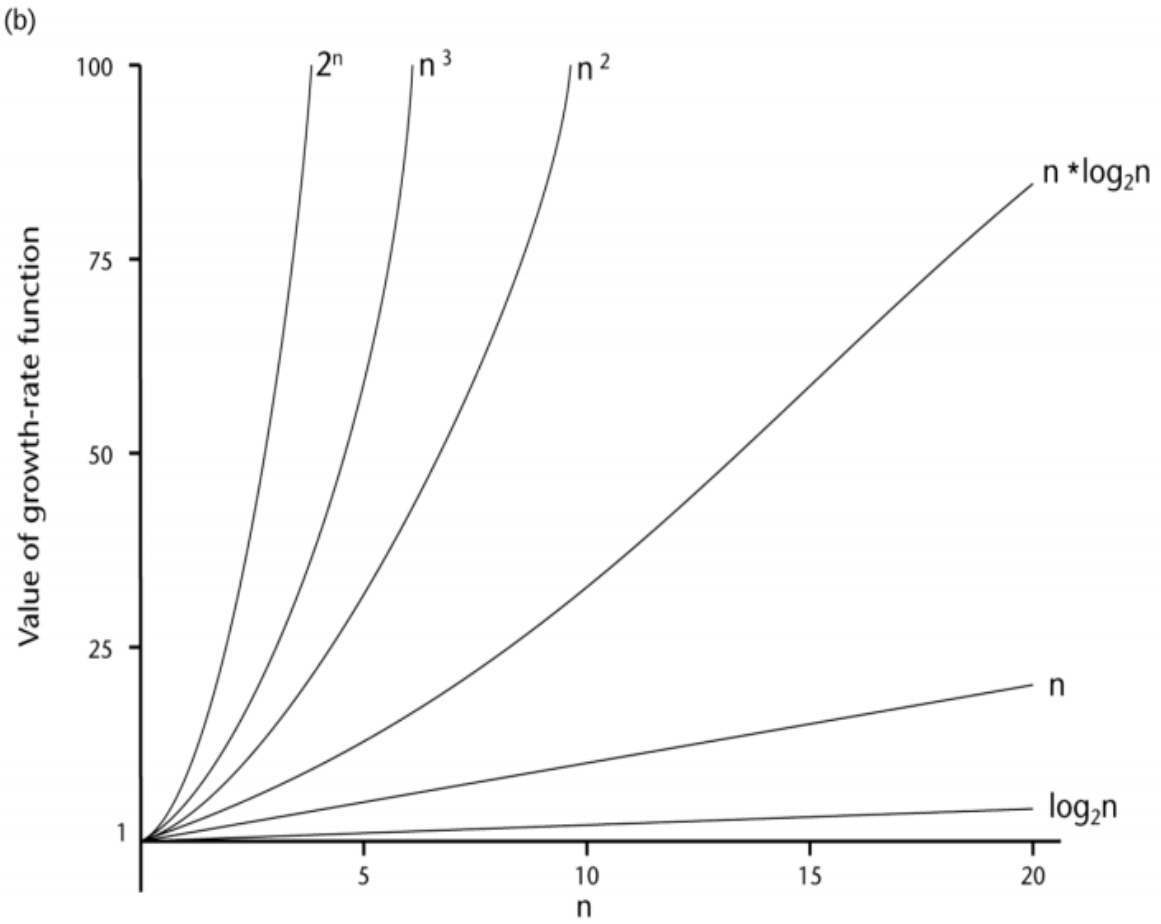




(Relations between $\theta, O, \Omega$)

The most common classes are given names:

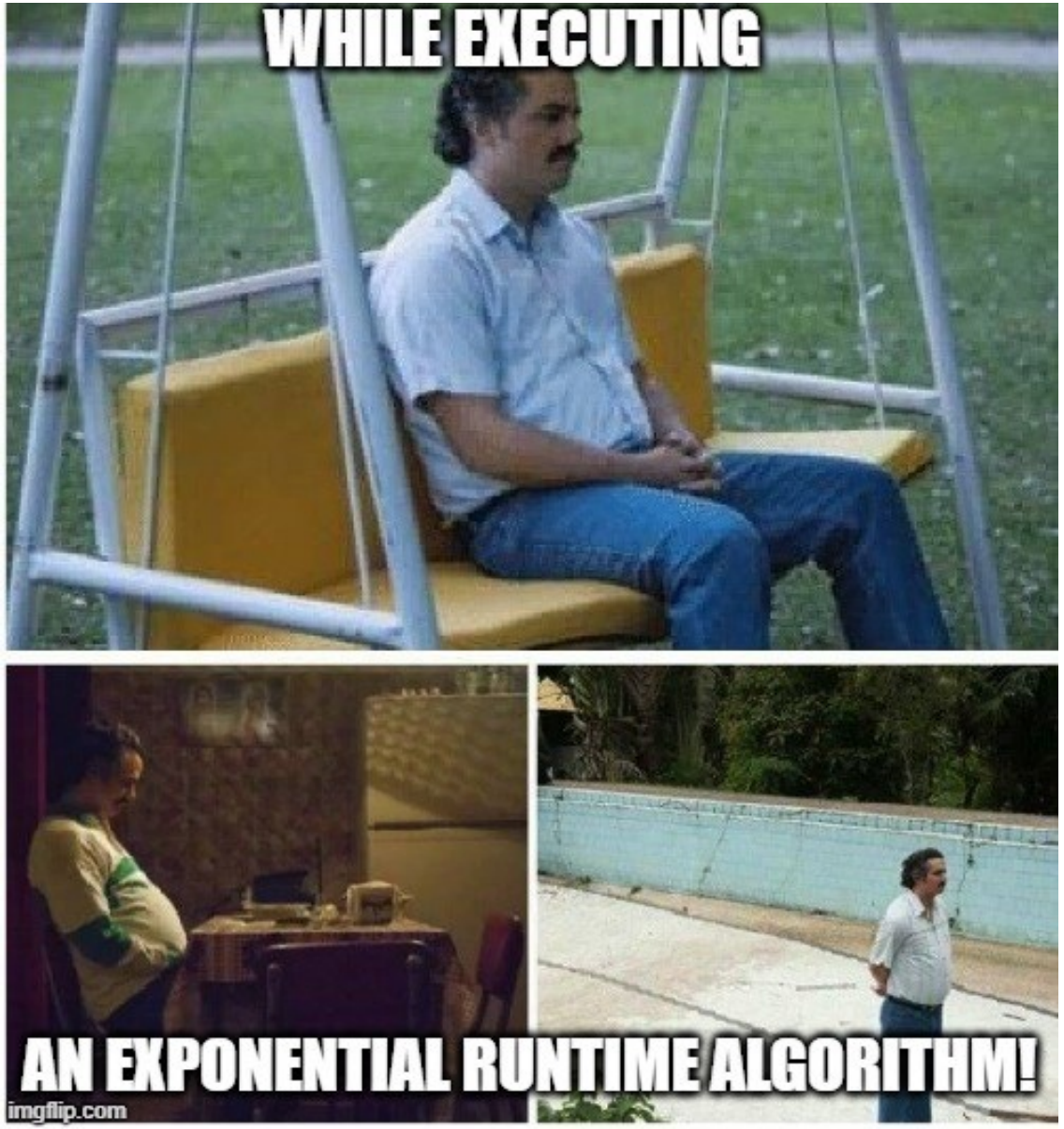
| | |
|---|---|
| $\Theta(1)$ | constant |
| $\Theta(\ln(n))$ | logarithmic |
| $\Theta(n)$ | linear |
| $\Theta(n \ln(n))$ | "$n$ log $n$" |
| $\Theta(n^2)$ | quadratic |
| $\Theta(n^3)$ | cubic |
| $2^n$, $e^n$, $4^n$, ... | exponential |




(Big-$\theta$ as an Equivalence Relation)

*Meme alert!*

# Try to keep the time-complexity simpler, else…



# Growth-Rate Functions

- If an algorithm takes *1 second to run with problem size = 8*, what is the time requirement (approximately) for with the problem size = 16 ?

| | $T(n)$ | $Time\ (seconds)$ |
|---|---|---|
| $O(1)$ | $1$ | $1\ second$ |
| $O(log_2^n) \sim O(logn)$ | $\frac{(1 \times log_2^{16})}{log_2^8} = \frac{4}{3}$ | $1.3333\ seconds$ |
| $O(n)$ | $\frac{(1 \times 16)}{8}$ | $2\ seconds$ |
| $O(nlogn)$ | $\frac{(1 \times 16 \times log_2^{16})}{8 \times log_2^8} = \frac{8}{3}$ | $2.6667\ seconds$ |
| $O(n^2)$ | $\frac{(1 \times 16^2)}{8^2}$ | $4\ seconds$ |
| $O(n^3)$ | $\frac{(1 \times 16^3)}{8^3}$ | $8\ seconds$ |
| $O(2^n)$ | $\frac{(1 \times 2^{16})}{2^8} = 2^8$ | $256\ seconds$ |

# Operators

- Each *machine instruction* can be executed in a fixed number of cycles,
  - We assume each *operation* requires a fixed number of cycles.
- Time required for any operator is $\theta(1)$.

| Operation type | Symbols |
|---|---|
| Retrieving/ storing variables from memory | |
| Variable assignment | = |
| Integer operations | + - * / % ++ -- |
| Bitwise operations | & \| ^ ~ |
| Relational operations | == != < <= > >= |
| Logical operations | && \|\| ! |
| Memory allocation and deallocation | new delete |

# Block of operations

- If each operation runs in $\theta(1)$ time, any fixed number of operations also run in $\theta(1)$ time.

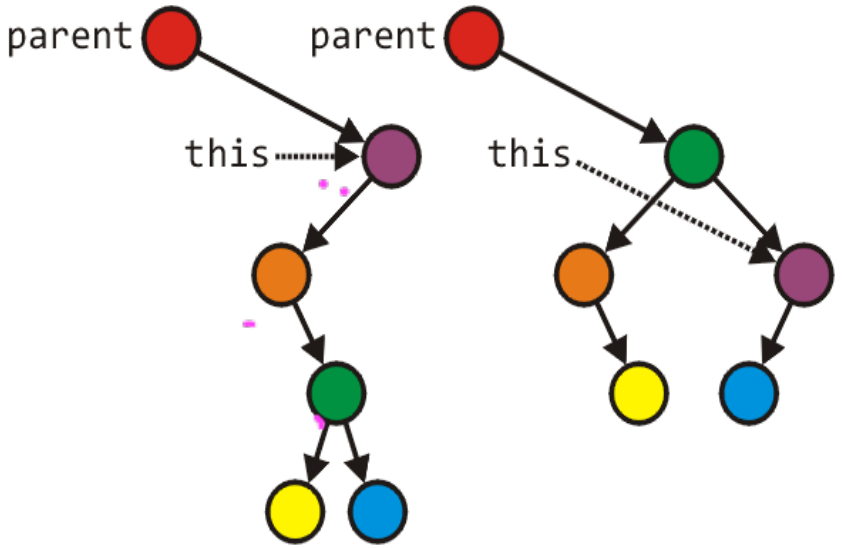
| | |
|---|---|
| `// swap variables a and b`<br>`int temp = a;`<br>`a = b;`<br>`b = temp;` | $\theta(1)$ |

| | |
|---|---|
| `// update a sequence of values`<br>`++index;`<br>`prev_modulus = modulus;`<br>`modulus = next_modulus;`<br>`next_modulus = modulus_table[index];` | $\theta(1)$ |

```
// This example rearranges an AVL
tree structure

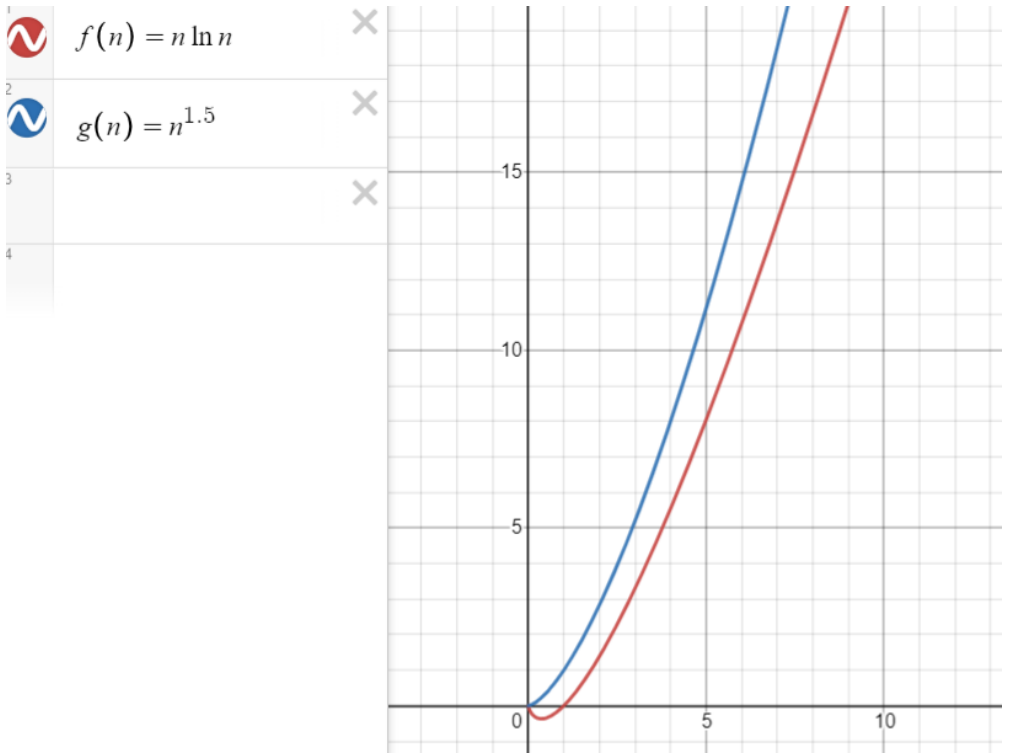
Tree_node *lrl = left->right->left;
Tree_node *lrr = left->right->right;
parent = left->right;
parent->left  = left;
parent->right = this;
left->right = lrl;
left = lrr;
```

$\theta(1)$



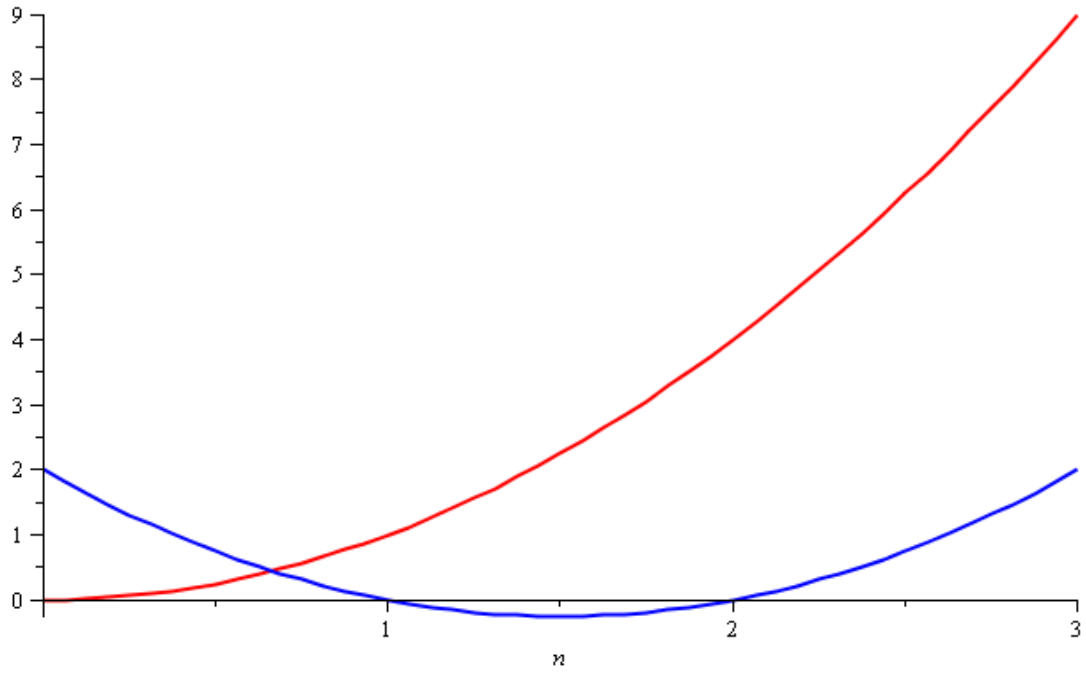
# Blocks in sequence

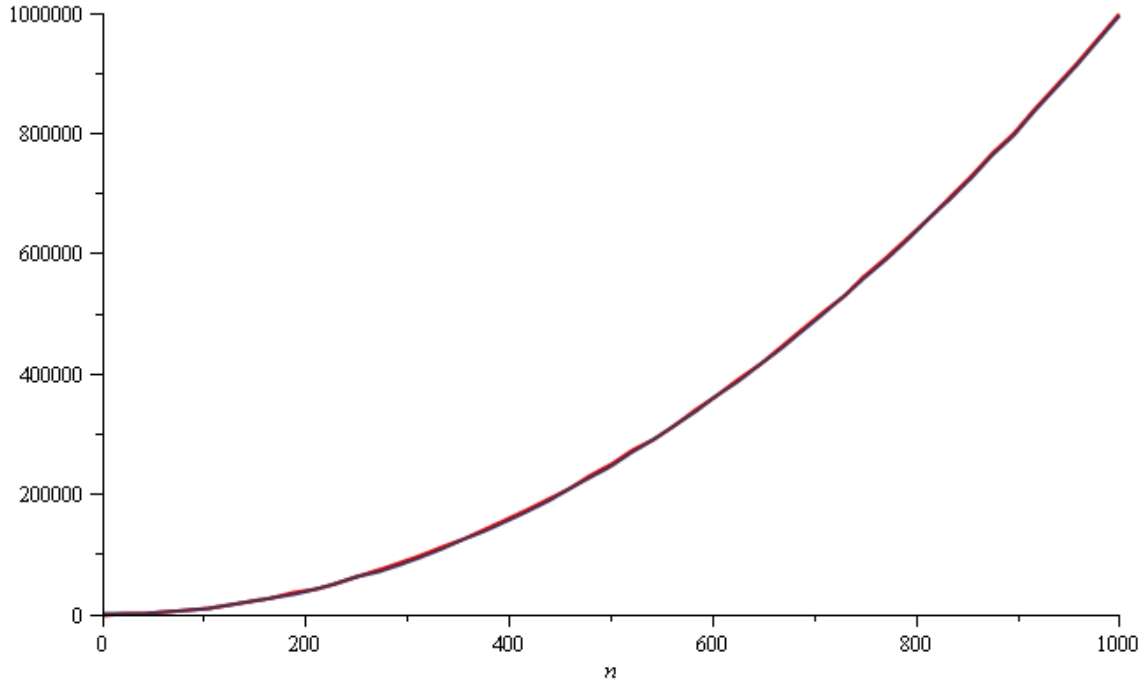
- Other examples include:
  - Run three blocks of code which are $\Theta(1), \Theta(n^2)\ and\ \Theta(n)$
    - Total runtime $\Theta(1 + n^2 + n) = \Theta(n^2)$
  - Run two blocks of code which are $\Theta(nlogn)\ and\ \Theta(n^{1.5})$
    - Total runtime $\Theta(nlogn + n^{1.5}) = \Theta(n^{1.5})$
  - While considering a sum, *take the dominant term.*



# Quadratic Growth

$$f(n) = n^2 \qquad g(n) = n^2 - 3n + 2$$



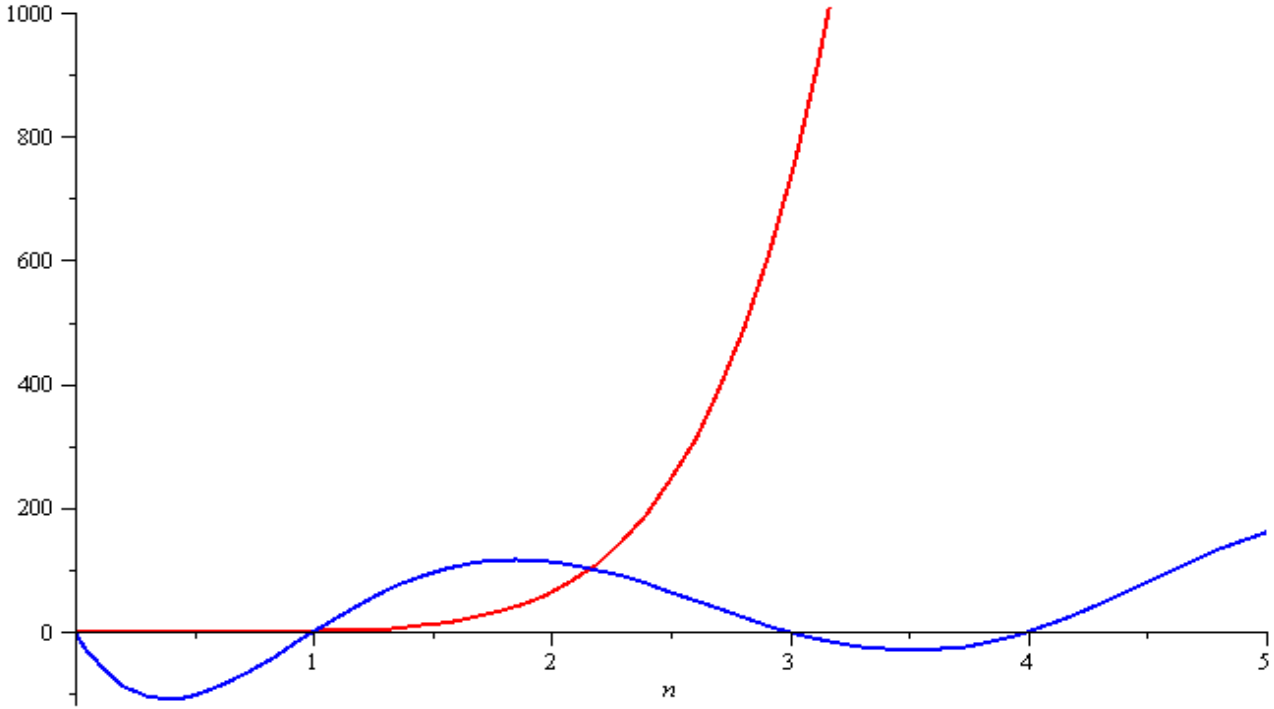

- Close to $n = 0$, they look very different
- Yet on the range $n = [0, 1000]$, they are (relatively) indistinguishable.

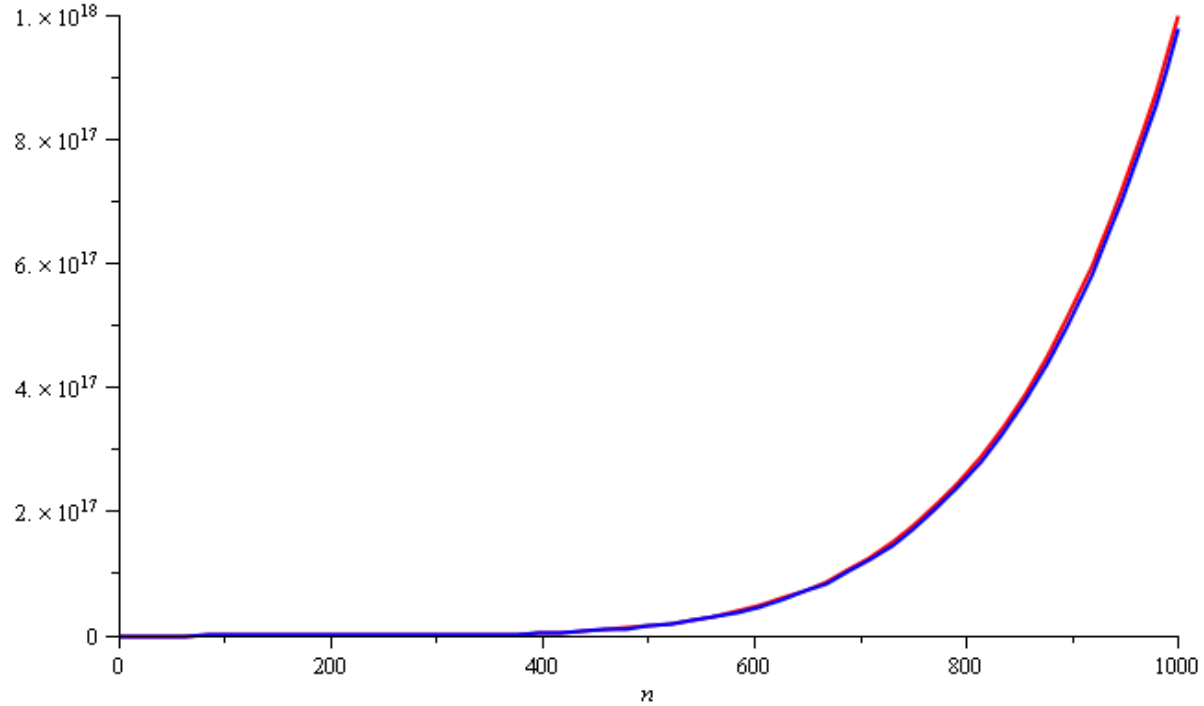
(continued from previous slide)

- Assuming $n = 1000$, the absolute difference is large, for example:
  - $f(1000) = 1{,}000{,}000$
  - $g(1000) = 997{,}002$
  - But the relative difference is very small:
    - $\left|\frac{f(1000)-g(1000)}{f(1000)}\right| = 0.002998 \sim 0.3\%$ (which is very small !)
    - And the difference goes to zero as $n \to \infty$

# Polynomial Growth

$$f(n) = n^6 \qquad g(n) = n^6 - 23n^5 + 19n^4 - 729n^3 + 126n^2 - 648n$$




- Around $n = 0$, they look very different
- Still, around $n = 1000$, the relative difference is less than $3\%$

- The polynomials acted similarly, since each had the same leading term.
  - $n^2$ in the first case, $n^6$ in the second.

- Suppose the coefficients of the leading terms were different:
  - In that case, *both functions would exhibit the same rate of growth*, however, *one would always be proportionally larger*.

# Control statements

Next we will look at the following control statements

- These are statements which *potentially alter the execution of instructions*
  - Conditional statements
    - `if, switch`
  - Condition-controlled loops
    - `for, while, do-while`
  - Count-controlled loops
    - `for i from 1 to 10 do … end do;`

| | | |
|---|---|---|
| `if (condition) {`<br>`  // true body`<br>`}`<br>`else {`<br>`  // false body`<br>`}` | Runtime of a conditional statement =<br>The runtime of the condition (the test) +<br>The runtime of the *body* | Mostly, runtime of the condition is $\theta(1)$<br>But runtime for body varies… |

# Condition controlled loops

```
for (int i=0; i<N; ++i) {
    // …
}
```

```
int i=0;                // initialization
while ( i<N ) {         // condition
    // …
    ++i;                // increment
}
```

- The initialization, condition-checking and increment statements are usually $\theta(1)$
- But repetitive condition checking increases overall cost !
- Assuming there are no break or return statements in the loop, the runtime is $\Omega(n)$

```
for (int i=0; i<N; ++i) {
    code which is θ(f(n));
}
```

Body repeatedly (n times) executes the $\theta(f(n))$ block.
Thus, $T(n) = \theta(nf(n))$
If, $f(n) = \theta(n^2)$,
Then, $T(n) = \theta(n \times n^2) = \theta(n^3)$

```
int sum = 0;
for (int i=0; i<N; ++i) {
    sum += 1; // θ(1)
}
```

$T(n) = \theta(n \times 1) = \theta(n)$

```
int sum = 0;
for (int i=0; i<n; ++i) {
    for (int j=0; j<n; ++j){
        sum += 1;    // θ(1)
    }
}
```

The inner loop is $\theta(n)$

Thus, $T(n) = \theta(n \times n) = \theta(n^2)$

```
// suppose with each loop, we use a linear
search over an array of size m:

for (int i=0; i<n; ++i) {
    search through an array of size m   // O(m)
}
```

The inner loop is $O(m)$

Thus, $T(n) = O(n \times m) = O(nm)$

```
1 int sum = 0;
2 for (int i=0; i<n; ++i) {
3     for (int j=0; j<i; ++j){
4         sum += i + j;
5     }
6 }
```

- To calculate the overall time complexity, first of all calculate the complexity of inner-loop (Line 3-5):

| | Cost for Inner loop (unit of time) = number of times it runs Line-4 + one extra condition checking when loop breaks | The inner loop runs |
|---|---|---|
| i=0 | 0 + 1 = 1 | (i+1) times |
| i=1 | 1 + 1 = 2 | (i+1) times |
| i=2 | 2 + 1 = 3 | (i+1) times |
| i=… | … | (i+1) times |
| i=n-1 | (n-1) + 1 = n | (i+1) times |

- So the runtime for *outer loop* can be summarized as:

$$\theta\left(1+\sum_{i=0}^{n-1}(i+1)\right)$$
$$=\theta\left(1+\sum_{i=0}^{n-1} i+\sum_{0}^{n-1} 1\right)$$
$$=\theta\left(1+\sum_{i=0}^{n-1} i+\mathrm{n}\right)$$
$$=\theta\left(1+n+\frac{n(n-1)}{2}\right)$$
$$=\theta\left(n^{2}\right)$$

```
switch (i) {
    case 1: /* do stuff */ break;
    case 2: /* do other stuff */ break;
    case 3: /* do even more stuff*/ break;
    case 4: /* tired yet? */ break;
    case 5: /* do stuff */ break;
    default: /* do default stuff */
}
```

```
if (i==1){ /* do stuff */ }
else if (i==2){ /* do other stuff */ }
else if (i==3){ /* do even more stuff*/ }
else if (i==4){ /* tired yet? */ }
else if (i==5){ /* do stuff */ }
else { /* do default stuff */}
```

- Switch statements appear to be nested if statements.
- Thus a switch statement would appear to run in $O(\#cases)$ time. (assuming each block takes $O(1)$ time to execute.

- Suppose we run one block of code followed by another block of code
  - Such code is said to be run *serially.*

- If the first block of code is $O(f(n))$ and the second is $O\big(g(n)\big)$, then the runtime of two blocks of code is $O(f(n) + g(n))$. (This usually simplifies to one or the other)

- As well as determining the runtime of an algorithm, if the data may not be deterministic, we may be interested in:
  - Best-case runtime
  - Average-case runtime
  - Worst-case runtime
- In many cases, they will be significantly different
- Example:
  - Searching a list linearly is simple enough
  - We will count the number of comparisons!

| | | |
|---|---|---|
| Best case | First element is what we are looking for | $O(1)$ |
| Worst case | Last element is what we are looking for | $O(n)$ |
| Average case | - We need some information about the list..<br>- Assuming, the number we are looking for *is in the list* and *equally likely distributed at any position.*<br>- If the size of list is n, there is as $1/n$ *chance* of it being in $i$-$th$ location | $\frac{1}{n}\sum_{i=1}^{n} i = \frac{1}{n}\left(\frac{n(n+1)}{2}\right)$<br>$= \frac{n+1}{2} = O(n)$ |

*Meme alert!*

# Planning to use linear search?



# Thank you